
Modelos de Clasificación

Aprendizaje de Máquinas, MDS (Primavera 2025)

Patricio Espinoza A.^{* 1} Francisco Vásquez L.^{* 1 2} Álvaro Márquez² Víctor Hernández M.³

Abstract

Este documento contiene la resolución de los problemas de la Tarea 3 del ramo Aprendizaje de Máquinas del Máster en Ciencia de Datos de la Universidad de Chile.

1. Pregunta 1

Parte (a)

El enfoque discriminativo modela directamente $P(y|x)$, por la cual se decide la asignación de clases mediante una frontera que maximiza la probabilidad condicional de las etiquetas de acuerdo a las características.

El enfoque generativo modela $P(x|y)$ y $P(y)$ para que, mediante Bayes, se obtenga $P(y|x)$. A diferencia del discriminativo, no existe una frontera sino que se modela la producción para cada clase dadas las características.

El enfoque discriminativo se expresa en la Regresión Logística ya que tiene una frontera mediante el uso de una función exponencial que modela la relación entre características y las clases objetivo. Por otra parte, el enfoque generativo se expresa en que mediante la Regresión Logística se puede obtener la sigmoide, que posteriormente da lugar a la softmax.

Parte (b)

De la Tabla 1 podemos ver que para la variable 'HeartDisease' tanto en los conjuntos Train como Test las distribuciones de las clases son prácticamente iguales, con 0.55% para la clase 1 y 0.44% para la clase 0, lo que coincide con las estadísticas de la Tabla 2, con medias y desviaciones estándar iguales entre ambos conjuntos. Además, de estos porcentajes podemos ver que hay un desbalance de clases mínimo.

En cuanto a la variable 'Cholesterol' podemos ver una gran cantidad de niveles en 0 para ambos conjuntos (Figura 1). Y que además, al omitir los valores 0, en el conjunto de Train se tiene una distribución similar a la normal, mientras que para el de Test esta es menos notoria, pero mantiene la concentración entre los niveles 200 a 300.

Al omitir los valores con 0, de la Tabla 3 podemos observar que ambas distribuciones tienen promedios y desviaciones estándar similares, pero varían notoriamente en los niveles mínimos y máximos, mostrando también una mayor concentración niveles 100-200 en Test que en Train (Percentil 25%).

Parte (c)

De la Tabla 4 podemos ver que al entrenar un modelo de Regresión Logística con split 80/20 se obtuvieron Accuracy (0.75 y 0.76) similares en ambos conjuntos, lo mismo para Precision (0.77 y 0.78) y F1-Score (0.78 y 0.79), indicando que el modelo tiene métricas generales aceptable de rendimiento.

Por otra parte, de la matriz de confusión de cada conjunto podemos ver que aunque dominan TP y TN, hay una cantidad considerable y similar de FP y de FN en ambos conjuntos.

En cuanto a las tasas, el FNR de ambos conjuntos es de alrededor de 0.2 (bajo), el FPR en ambos es alrededor de 0.28 (ligeramente más alto), mientras que el Recall tuvo buen desempeño en ambos conjuntos (0.78 y 0.80), al igual que la Specificity (0.72 y 0.72). Estas métricas nos muestran un buen desempeño general del modelo con una buena capacidad de generalización.

Desde un enfoque discriminativo, la alta especificidad y baja sensibilidad (alto recall) muestran una buena frontera y trade-off del umbral.

Mientras que desde un enfoque generativo, si los supuestos se aplican, entonces podemos sugerir que un modelo LDA o Gaussian NB puede dar mejores probabilidades y, a veces, mejor rendimiento con menos datos de los usados para la Regresión Logística.

Parte (d)

Al cambiar los split de entrenamiento y prueba a 60/40 podemos ver que el FNR aumenta ligeramente en ambos conjuntos, mientras que el FPR disminuye. Asimismo, la Recall disminuye mientras que la Especificidad aumenta. En cuanto a las métricas, la Accuracy se mantuvo, y la Precision aumento solo en el conjunto de Test, aunque para

ambos se obtuvo una menor F1-Score que con el split inicial.

En resumen, el cambio de split provoca un aumento de Falsos Negativos y una disminución de Falsos Positivos. Es decir, el modelo ajustó la frontera a una más rígida debido a tener menos ejemplos con los que aprender y perdió una ligera capacidad de generalización.

Parte (e)

En problemas de clasificación binaria la curva ROC muestra TPR vs FPR al variar el umbral de corte, indicando la capacidad del modelo para discriminar entre clases a través de todos los umbrales. Por otra parte, el área bajo la curva ROC (AUC) mide la probabilidad de que el modelo asigne un score mayor a una muestra positiva que a una negativa.

De las Figuras 2 y 3 podemos ver que al disminuir el conjunto de entrenamiento y aumentar el de prueba, AUC se reduce, indicando una menor capacidad de discriminación entre clases, aún así, las capacidades de discriminación de ambos modelos se consideran fuertes, con valores sobre 0,8. También por el tipo de Curva ROC, podemos notar que la de un conjunto de entrenamiento de 80% crece más rápido, indicando que resulta en un mejor modelo.

Parte (f)

La función KFold divide en k pliegues, sin mantener proporciones de clases, lo cual es útil para datos balanceados.

StratifiedKFold por otra parte, es igual que KFold pero preserva la proporción de clases en cada pliegue, lo que lo hace recomendable para clasificación con clases desbalanceadas.

RepeatedKFold realiza repeticiones de KFold varias veces con distintos shuffles, reduciendo la varianza de la estimación.

Por otra parte, en LeaveOneOut (LOO) cada observación es test una vez, lo que lo hace exhaustivo pero computacionalmente caro. Resulta útil para datasets muy pequeños.

De igual forma, en LeavePOut (LPO) se prueban todas las combinaciones posibles dejando p observaciones fuera, resultando también costoso.

Con los parámetros $n_splits = 5$, $shuffle = True$, para KFold y StratifiedKFold, y además $n_repeats = 2$ para RepeatedKFold. Junto a usando una submuestra del 10% del dataset y $p=2$ para LeavePOut. Se obtuvieron los resultados de la Tabla 6.

Para Accuracy se puede ver que KFold, StratifiedKFold y RepeatedKFold destacan con 0.76, seguida de LeaveOneOut con 0.75 y LeavePOut con 0.70. Este mismo patrón se repite para F1-Score, con la diferencia de que el desempeño de LeaveOneOut y LeavePOut fue mucho peor (0.43 y 0.60 respectivamente). Dada la formulación de estos últimos

dos (una y dos muestras a evaluar que no permiten obtener TPR y FPR en cada simulación de prueba), no se obtuvieron valores para ROC-AUC, mientras que para KFold, StratifiedKFold y RepeatedKFold todas estuvieron en torno a 0.84.

F1-Score permite ver de forma general la matriz de confusión, y es recomendable ya que a accuracy puede estar sesgada para una clase mayoritaria. En base a esto, el método de validación cruzada recomendado sería StratifiedKFold por su robustez ante desbalances, y pese a que esto no destaca en nuestro dataset, si puede introducir ruido en las métricas medidas, lo que queda demostrado al compararlas con las demás y ver un mayor puntaje. En base a los resultados, los modelos tienen una buena generalización al ser entrenados de forma diversa pero equitativamente distribuida.

2. Pregunta 2

Parte (a)

Al generar las variables dummies para las categóricas se puede ver a partir de la Tabla 7 que la Clase 0 corresponde al 81.74% de los datos, mientras que la Clase 1 al 18.26%.

Parte (b)

En el contexto de la función LogisticRegression, el parámetro *penalty* indica el tipo de regularización que se aplicará. 'l1' es Lasso, la cual permite que los coeficientes se vayan a cero y por tanto permita seleccionar variables. Mientras que 'l2' es Ridge, la cual penaliza la magnitud de los coeficientes haciéndolos más pequeños pero no necesariamente cero. Además 'elasticnet' mezcla tanto Lasso como Ridge.

Por otra parte, el parámetro C indica el inverso de la fuerza de regularización. Un valor pequeño implica una regularización fuerte y por tanto coeficientes más pequeños (mayor sesgo, menos varianza). Un valor grande de C implica menos regularización, coeficientes más grandes y mayor flexibilidad.

Finalmente, el parámetro *class_weight* aborda el desbalance de clases, 'balanced' ajusta los pesos automáticamente de forma inversa a la frecuencia de las clases. También se pueden ajustar los pesos manualmente para las clases y con 'None' se omite cualquier procedimiento. Este parámetro afecta a la función objetivo, controlando el trade-off entre precision y recall.

De forma general, *penalty* y C controlan el complexity y la magnitud de β , mientras que *class_weight* redefine qué errores son más costosos.

Parte (c)

A partir de la Tabla 8, podemos observar que al probar con los parámetros $\text{penalty} \in \{l1, l2, \text{elasticnet}\}$, $C \in \{0.1, 1, 10\}$ y $\text{class_weight} \in \{\text{balanced}, \text{None}, \{0 : 1, 1 : 5\}\}$, el mejor resultado tomando como métrica el AUC se obtuvo para $C = 10$, $\text{class_weight} = \text{None}$ y $\text{penalty} = l1$, con un valor de $AUC = 0.657$, $\text{Accuracy} = 0.818$ y $F1 = 0.011$.

Por otro lado, al guiarnos por la métrica F1, un resultado más equilibrado se obtuvo con $AUC = 0.655$, $\text{Accuracy} = 0.592$ y $F1 = 0.366$, utilizando los parámetros $C = 1$, $\text{class_weight} = \text{balanced}$ y $\text{penalty} = l1$.

En general, como referencia, el máximo valor de AUC alcanzado fue 0.657, la Accuracy máxima fue 0.818 y el F1 máximo fue 0.366. Esto refleja un claro desbalanceo de clases, ya que se obtiene una Accuracy alta pero un valor de F1 bajo.

Parte (d)

De la Tabla 9 podemos ver que muchos modelos muestran $\text{Accuracy} = 0.8174$ y $F1 = 0.0$, indicando que el clasificador posiblemente está prediciendo siempre la clase mayoritaria, lo que podemos comprobar al ver la distribución de la Tabla 7.

También de la Figura 4 podemos ver que L2 tiene la menor media de las normas con $\mu = 2.25$, ascendiendo con Elasticnet con $\mu = 2.29$ y L1 con $\mu = 2.36$. De la Figura 5 se observa que a medida que aumenta 'C' también lo hace la norma, y de la Figura 6 se observa que la distribución de pesos 0:1, 1:5 penaliza más los errores en la clase minoritaria generando una media de norma mayor ($\mu = 2.42$ vs $\mu = 2.21$ y $\mu = 2.28$) en comparación de pesos balanceados o sin pesos.

En resumen, se tiene que para un C pequeño, la regularización es fuerte, se penalizan coeficientes grandes y la norma es pequeña. Mientras que para un C grande, la regularización es débil, existen coeficientes grandes y por tanto la norma es grande.

Por otro lado, la penalty L1 (Lasso) tiende a generar muchos coeficientes exactamente en cero, generando una menor norma. Mientras que L2 (Ridge) reduce las magnitudes sin que sean cero, y la norma es intermedia. En general se tuvo el comportamiento de normas $\hat{B}_M^{l1} > \hat{B}_M^{\text{elasticnet}} > \hat{B}_M^{l2}$, con excepciones dependiendo de otros hiperparámetros (Tabla 9).

Finalmente, class_weight permite cambiar la ponderación de las clases al modificar la función de pérdida y lograr que el modelo aumente la magnitud de algunos coeficientes para compensar el desbalance, resultando en mayores normas como sucede con 0:1, 1:5.

Parte (e)

A partir de la Tabla 10 se puede observar que la variable Crowdedness es la que más influencia tiene en la predicción, con un $|\beta_m| \in [0.85, 0.94]$.

Esto indica que, en la dirección de los parámetros aprendidos, Crowdedness domina la orientación del hiperplano de decisión, es decir, es la característica que más influye en la probabilidad predicha por el modelo. Además se observa un patrón con los hiperparámetros, donde configuraciones con C más grandes (menor regularización) y penalizaciones no balanceadas ($\text{class_weight} = \text{None}$) tienden a producir los $|\hat{\beta}_m|$ más altos.

Una combinación de hiperparámetros que tiende a reducir $|\hat{\beta}_m|$ para Crowdedness es valores pequeños de C (regularización más fuerte) y algunas configuraciones con class_weight (una aparente tendencia por las balanced) reducen ligeramente la magnitud de $|\hat{\beta}_m|$. Sin embargo, ninguna cambia la variable predictora predominante.

A partir del Histograma de la Figura 7 se puede ver que los 3 tipos de penalización tienen valores cercanos a 0, en particular, L1 tiene gran presencia en aquellos con 0 y también alcanza los mayores valores de $|\hat{\beta}_m|$. En cuanto a C, se observa que para valores altos de $|\hat{\beta}_m|$ predomina C=1 y C=10. Mientras que para los pesos de las clases se ve una distribución más equitativa en 0 y en valores altos.

Parte f

La mejor combinación de hiperparámetros resultó para maximizar $|\hat{\beta}_m|$ fue $\text{penalty} = l1$, $C=1.0$ y $\text{class_weight} = \text{None}$. Sin embargo, para maximizar F1 la mejor combinación es L2, $C = 1.0$ y $\text{class_weight} = \text{balanced}$ con un AUC de 0.68, Accuracy de 0.62 y F1 de 0.3919. Si bien hay otras con Accuracy o AUC mayor, esta rescata que predice correctamente algunas instancias de la clase minoritaria. El parámetro C provoca una regularización intermedia, que junto con L2 (Ridge) permite enfocarse en las variables predictoras útiles. Asimismo, el peso balanceado permite abordar el problema de la clase minoritaria y las fallas en predecirla.

3. Pregunta 3

Parte (a)

De la Tabla 11 se puede observar que en ambos conjuntos la variable objetivo tiene una distribución similar, con 55% para la Clase 1 y 44% para la Clase 2. Esto indica que el desbalanceo entre clases es mínimo.

Parte (b)

Cada uno de los modelos usados tiene los siguientes enfoques y limitaciones:

La Regresión Lógica es discriminativa linealmente, y espera relaciones lineales logit, funcionando bien como sucedió según la Tabla 12, donde sus resultados fueron Train Acc 0.8571, Test Acc 0.8696, Test AUC 0.9407, Test F1 0.8800. Indicándonos que tuvo buen rendimiento y buena generalización. Asimismo, la alta AUC indica que el modelo ordena bien positivos/negativos. Su principal limitación es ante relaciones no lineales como lo serían las polinómicas o periódicas.

El Perceptron obtuvo los resultados: Train Acc 0.6283, Test Acc 0.6522, Test AUC 0.8211, Test F1 0.5789, Test Recall 0.4314 (bajo). Tuvo el peor desempeño, con una alta precisión pero Recall bajo, es decir predice pocos falsos positivos, pero muchos falsos negativos. Esto se puede deber a que es un clasificador lineal más sencillo, que es sensible a la escala y al ruido de los datos, además de que no produce probabilidades ni regularización por defecto. Su limitación es que ante la falta de escalado y regularización dio un peor desempeño.

KNN obtuvo los siguientes resultados: Train Acc 0.7845, Test Acc 0.6848, Test AUC 0.7468, Test F1 0.7129. Se destaca que tuvo una pérdida de rendimiento entre test vs train, indicando cierto sobreajuste. Por otra parte, tuvo buen recall en train, que cae en test. Esto muy probablemente se debe a que no es paramétrico, y es sensible a las escalas y ruido de los datos. Su limitación radica en que necesita vecinos representativos, y es muy sensible a la maldición de la dimensionalidad.

Por otra parte, la GaussianNB (generativo) obtuvo: Train Acc 0.8644, Test Acc 0.8804, Test AUC 0.9536 (mejor AUC). Este buen desempeño se puede atribuir a que probablemente las características numéricas separan bien las clases, lo que va de la mano con que la GaussianNB asume independencia condicional por característica y distribuciones gaussianas, lo que debido al buen rendimiento nos indicaría que hay datos que siguen estos supuestos.

Por último, el LDA tuvo los resultados: Train Acc 0.8535, Test Acc 0.8043, Test AUC 0.9225. Tuvo un buen rendimiento pero también una mayor caída al test respecto a GaussianNB/Regresión Logística. El AUC alto también

indica una buena capacidad del modelo. Esto se debe a que LDA modela gaussianas por clase con igual covarianza, lo que a su vez lo deja limitado a este supuesto, con sensibilidad a la desigualdad de covarianzas entre clase.

En base a los conocimientos del curso, posibles mejoras para los modelos pueden ser el preprocesamiento con escalamiento (Para el Perceptron), búsqueda de hiperparámetros óptimos para cada modelo, ver si el pequeño desbalance de clases afecta, y en dicho caso tratarlo con técnicas de Sampling. Además, aunque también se pueden complejizar las predicciones con polinomios o sinusoidales, estas no se adaptan al contexto de nuestros datos.

Parte (c)

A partir de los valores AUC de la Figura 8 podemos determinar que los modelos que más se acercan a un clasificador perfecto en orden de mejoría son KNN → Perceptron → LDA → LogisticRegression → GaussianNB.

Estos nos muestra claramente como los modelos menos perfectos son aquellos más sensibles al escalamiento, mientras que aquellos más centrados en distribuciones Gaussianas fueron mejores (LDA y Gaussian) o LR con las relaciones lineales, son capaces de maximizar AUC ya que distinguen correctamente entre clases.

Además, de las curvas ROC podemos ver que el crecimiento rápido al inicio indica una alta tasa de verdaderos positivos (TPR alto) con pocos falsos positivos (FPR bajo). Es decir, una distinción clara entre clases, con predicciones positivas correctas en su mayoría y un mejor desempeño general. Aquí se mantiene el mismo orden mencionado anteriormente, destacando como KNN y Perceptron no se ajustan bien como clasificadores debido a sus limitaciones (Parte a).

Parte (d)

De la Tabla 13, podemos ver que los modelos con mejores métricas globales son GaussianNB, LogisticRegression y LDA, especialmente cuando se usan todas las variables ($AUC \approx 0.96-0.97$, $Accuracy \approx 0.92-0.91$, $F1 \approx 0.92-0.93$). KNN tiene buen desempeño con sólo variables categóricas ($AUC \approx 0.90$, $F1 \approx 0.86$) pero empeora con sólo variables numéricas ($AUC \approx 0.75$). El Perceptron muestra el peor comportamiento, sobre todo al usar todas las variables, donde F1 cae a 0.15, lo que sugiere problemas por falta de escalado, regularización o por alta dimensionalidad al mezclar dummies y continuas.

El modelo GaussianNB, que supone gaussianas por variables e independencia condicional obtuvo la mejor AUC al usar todas las variables (0.9665). Esto sugiere que las variables numéricas tienen distribuciones cercanas a gaussianas por clase y/o son informativas de forma aproximadamente independiente. Este comportamiento es aprovechado por el

modelo resultando en un buen rendimiento.

LogisticRegression que es discriminativo y con fronteras lineales tuvo buenos resultados con numéricas y con todas las variables ($AUC \approx 0.94-0.96$). Esto indica que la separación entre clases es en gran medida lineal o similar en el espacio de las características, lo que resultaría en un buen rendimiento.

LDA que es generativo con gaussianas y covarianza común, también fue bueno con las variables numéricas y con todas las variables ($AUC \approx 0.94-0.96$). Si las covarianzas por clase son similares, entonces su rendimiento alto apoya esta suposición y explica el por qué de su buen rendimiento.

KNN que es no paramétrico presenta de las métricas más bajas AUC (0.7468), Accuracy (0.6848) y F1 (0.7129) para los conjuntos numéricos y de todas las variables. Este rendimiento inferior al resto probablemente se deba por falta de escalado o porque las distancias no separan tan bien las clases.

Perceptron que es un clasificador lineal online y sin regularización por defecto fue el de peor rendimiento, especialmente al mezclar todas las variables con $F1 = 0.1455$. Esto nos indicaría sensibilidad al escalado o falta de regularización resultantes en el mal rendimiento.

Parte (e)

El escalamiento RobustScaler consiste en transformar las variables numéricas restando la mediana y dividiendo por el rango intercuartílico ($IQR = Q3 - Q1$). Centrando cada característica en 0 y reescalando de manera que valores típicos quedan dentro de un rango reducido. Es robusto frente a outliers ya que no depende de la media ni de la desviación estándar.

El escalamiento StandardScaler transforma cada variable restando su media y dividiendo por su desviación estándar, obteniendo una distribución con media 0 y varianza 1. Se adecua para modelos lineales o basados en distancia (KNN o Perceptron), que asumen que las variables están en escalas comparables y centradas.

Finalmente, el método quantile_transform aplica una transformación no lineal que mapea los cuantiles de cada variable a una distribución normal estándar, tal que las variables son aproximadamente gaussianas. Se adecua para modelos que suponen normalidad (LDA o GaussianNB), pero también puede distorsionar la relación lineal entre variables.

De la Tabla 14 podemos ver que, al aplicar un escalamiento evitando Data Leakage para el conjunto de todas las variables, las métricas mejoraron notablemente en los modelos sensibles a la escala de las variables, como KNN y Perceptron.

En KNN, el AUC aumentó de 0.75 (sin escalado) a 0.96

con el escalado quantile_transform. Para el Perceptron, el AUC pasó de 0.81 a 0.92 con StandardScaler. En cambio, modelos como GaussianNB, LogisticRegression y LDA muestran cambios mínimos tras el escalado, ya que son menos sensibles a la magnitud de las características o ya funcionan sobre supuestos gaussianos, aun así destaca que estos GNB y LDA aumentan con quantile_transform.

En cuanto a F1 podemos ver que KNN pasó de un F1 de 0.71 (sin escalado) a 0.94 con el escalado quantile_transform, mostrando el mayor aumento relativo, lo que confirma su sensibilidad a las distancias entre características. El Perceptron también presenta una mejora, subiendo de 0.15 (sin escalado) a aproximadamente 0.82 con StandardScaler, evidenciando que la normalización estabiliza su convergencia. En cambio, modelos como GaussianNB, LogisticRegression y LDA muestran cambios mínimos. Se aprecia que GaussianNB y LDA alcanzan sus mejores valores con el escalado Robust o Quantile Normal, lo que sugiere una ligera ganancia al suavizar el efecto de outliers o distribuciones asimétricas en los datos numéricos.

En cuanto a aplicar escalamiento sobre variables binarias, esto no tiene sentido ya que éstas solo toman valores 0 o 1. Escalarlas no aporta información adicional e incluso puede introducir valores intermedios sin interpretación.

Parte (f)

El mejor modelo sin escalado es Gaussian Naive Bayes ya que para el conjunto de todas las variables obtuvo la mayor AUC (0.9665) junto a una buena Accuracy (0.9130) y F1 (0.9184). Esto indica que las distribuciones condicionales de las variables numéricas por clase se aproximan razonablemente a gaussianas y que la independencia condicional no rompe el rendimiento del modelo. Al ser un modelo generativo con supuestos claros, explica por qué, aun sin escalado, captura bien la separación entre clases en este dataset.

Por otra parte, el mejor modelo con escalado es KNN ya que tras esta aplicación, alcanza la AUC más alta observada (0.9668) y también la F1 más alta (0.9412) con quantile_transform. Esta mejora se explica por la dependencia de KNN a las distancias, en donde el escalado las vuelve representativas y permite explotar la estructura local de los datos.

Si bien estos dos modelos para cada contexto son los que tuvieron mejores métricas. Opciones como Logistic Regression que también tuvo AUC y F1 altas puede resultar útil por la interpretabilidad de los coeficientes lineales. Esto podría ser especialmente necesario en nuestro caso, donde trabajamos con datos clínicos.

A. Anexo

A.1. Pregunta 1

A.1.1. PARTE A

Table 1. Distribución de la variable *HeartDisease* en los conjuntos de entrenamiento y prueba

HeartDisease	Proporción en y_{train}	Proporción en y_{test}
1	0.553	0.554
0	0.447	0.446

Table 2. Estadísticas descriptivas de la variable *HeartDisease* en los conjuntos de entrenamiento y prueba

Estadística	y_{train}	y_{test}
count	734	184
mean	0.5531	0.5543
std	0.4975	0.4983
min	0.0	0.0
25%	0.0	0.0
50%	1.0	1.0
75%	1.0	1.0
max	1.0	1.0

A.1.2. PARTE B

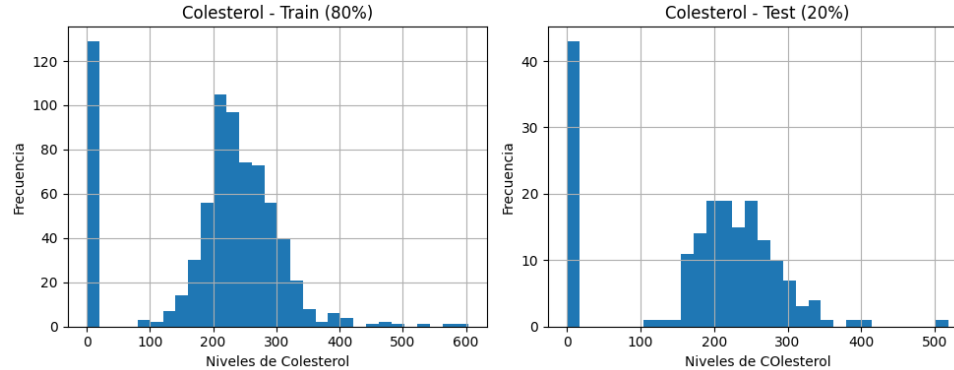


Figure 1. Distribución de los niveles de colesterol para los conjuntos Train y Test.

Table 3. Estadísticas descriptivas de la variable *Cholesterol* (sin valores 0) en los conjuntos de entrenamiento y prueba

Estadística	X_{train}	X_{test}
count	605	141
mean	246.56	236.37
std	59.65	56.40
min	85	110
25%	210	196
50%	238	233
75%	277	264
max	603	518

A.1.3. PARTE C

Table 4. Regresión Logística en los conjuntos de entrenamiento (80%) y prueba (20%)

Métrica	Train (80 %)	Test (20 %)
<i>Tasas</i>		
False Negative Rate (FNR)	0.217	0.196
False Positive Rate (FPR)	0.277	0.280
Recall (Sensibilidad)	0.783	0.804
Specificity (Especificidad)	0.723	0.720
<i>Métricas de desempeño</i>		
Accuracy	0.756	0.766
Precision	0.778	0.781
F1-score	0.780	0.792
<i>Conteos</i>		
True Positives (TP)	318	82
False Positives (FP)	91	23
True Negatives (TN)	237	59
False Negatives (FN)	88	20

A.1.4. PARTE D

Table 5. Regresión Logística en los conjuntos de entrenamiento (60%) y prueba (40%)

Métrica	Train (60 %)	Test (40 %)
<i>Tasas</i>		
False Negative Rate (FNR)	0.227	0.240
False Positive Rate (FPR)	0.264	0.226
Recall (Sensibilidad)	0.773	0.760
Specificity (Especificidad)	0.736	0.774
<i>Métricas de desempeño</i>		
Accuracy	0.756	0.766
Precision	0.783	0.807
F1-score	0.778	0.783
<i>Conteos</i>		
True Positives (TP)	235	155
False Positives (FP)	65	37
True Negatives (TN)	181	127
False Negatives (FN)	69	49

A.1.5. PARTE E

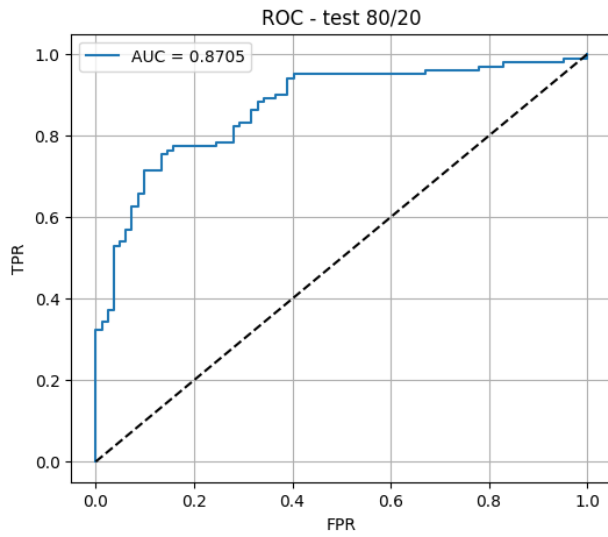


Figure 2. Curva ROC para el modelo con división 80% entrenamiento y 20% prueba.

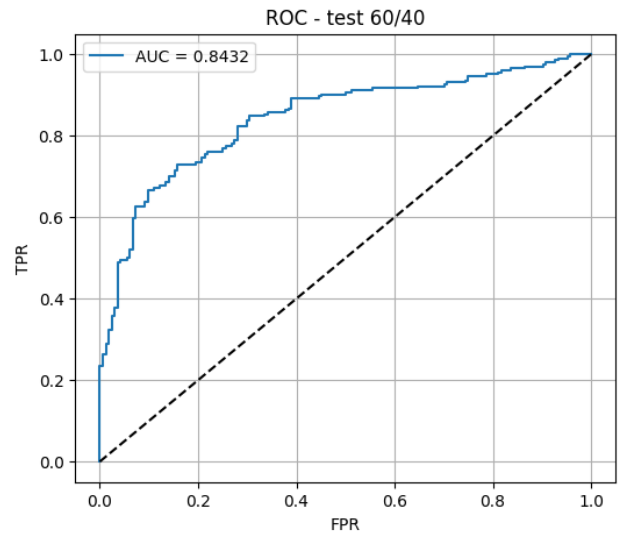


Figure 3. Curva ROC para el modelo con división 60% entrenamiento y 40% prueba.

A.1.6. PARTE F

Table 6. Resultados de validación cruzada con distintos métodos de validación cruzada

Método	Accuracy (mean $\pm$ std)	ROC AUC (mean $\pm$ std)	F1-score (mean $\pm$ std)
KFold	0.760 $\pm$ 0.050	0.842 $\pm$ 0.044	0.780 $\pm$ 0.055
StratifiedKFold	0.761 $\pm$ 0.030	0.842 $\pm$ 0.032	0.785 $\pm$ 0.024
RepeatedKFold	0.760 $\pm$ 0.041	0.841 $\pm$ 0.035	0.781 $\pm$ 0.044
LeaveOneOut	0.757 $\pm$ 0.429	—	0.434 $\pm$ 0.496
LeavePOut (10%)	0.700 $\pm$ 0.322	—	0.607 $\pm$ 0.422

A.2. Pregunta 2

A.2.1. PARTE A

Table 7. Distribución de variable objetivo.

Clase	Conteo	Porcentaje (%)
0	6266	81.74
1	1399	18.26
Total	7665	100.00

A.2.2. PARTE C

Table 8. Top 10 configuraciones por AUC en la búsqueda de hiperparámetros

C	class_weight	penalty	mean_test_AUC	std_test_AUC	mean_test_Accuracy	mean_test_F1
10	None	l1	0.657	0.010	0.818	0.011
1	None	l1	0.657	0.010	0.818	0.006
10	None	elasticnet	0.657	0.010	0.818	0.011
10	None	l2	0.657	0.010	0.818	0.011
1	None	elasticnet	0.656	0.010	0.818	0.006
1	None	l2	0.656	0.010	0.818	0.006
0.1	{0:1, 1:5}	l1	0.655	0.011	0.525	0.359
1	balanced	l1	0.655	0.012	0.592	0.366
10	balanced	l1	0.655	0.012	0.592	0.365
10	{0:1, 1:5}	l1	0.655	0.012	0.533	0.362

Mejor combinación (por AUC): {'C': 10, 'class_weight': None, 'penalty': 'l1'}, con AUC = 0.657.

Espacio de búsqueda: penalty $\in \{l1, l2, elasticnet\}$, C $\in \{0.1, 1, 10\}$, class_weight $\in \{balanced, None, \{0:1, 1:5\}\}$.

Mejor métricas: AUC: 0.657, Accuracy: 0.817 y F1: 0.366.

A.2.3. PARTE D

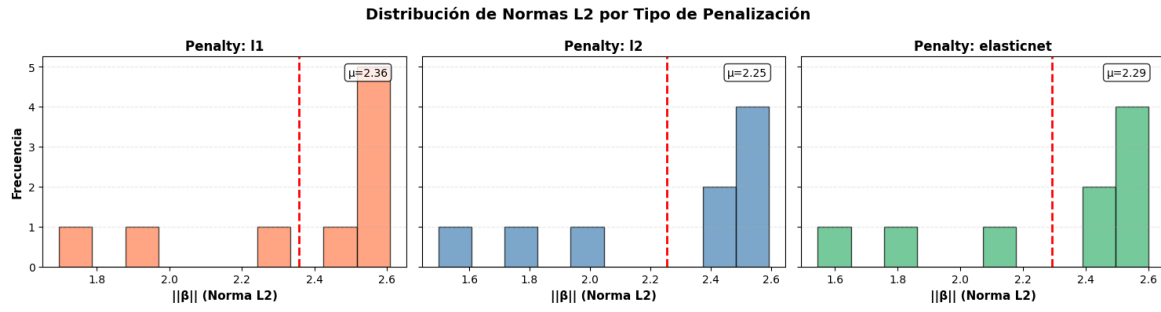


Figure 4. Distribución de $\hat{\beta}_M$ por tipo de penalización sin validación cruzada y con split 80%/20%

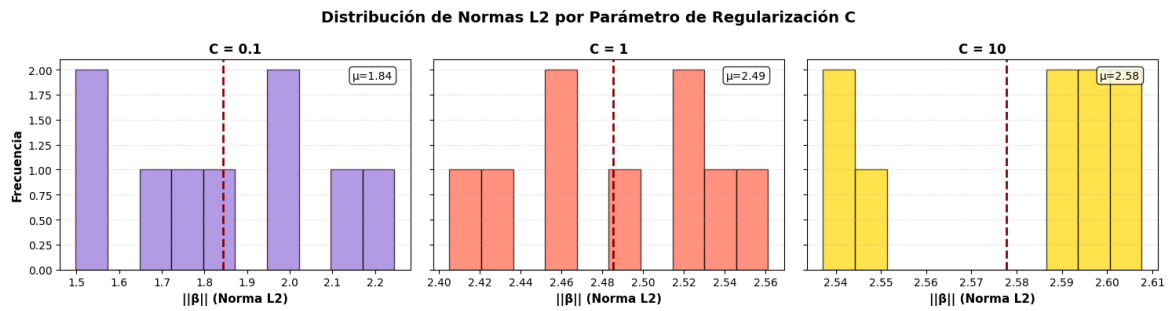


Figure 5. Distribución de $\hat{\beta}_M$ por tipo de regularización C sin validación cruzada y con split 80%/20%

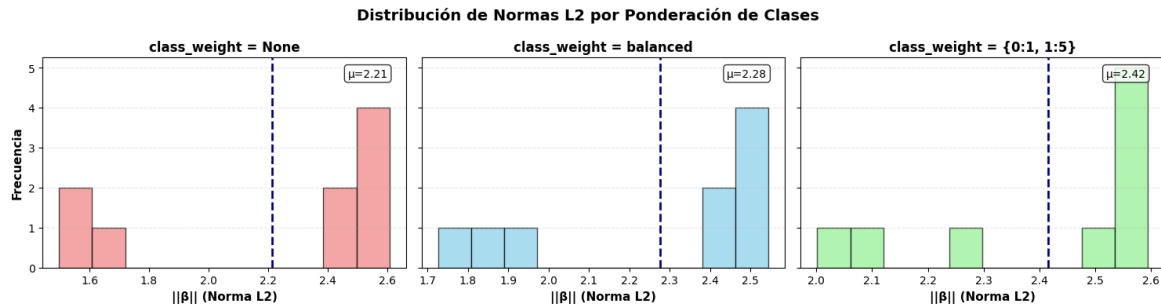


Figure 6. Distribución de $\hat{\beta}_M$ por tipo de balanceo de clases sin validación cruzada y con split 80%/20%

Table 9. Resultados de validación para 27 distintas configuraciones del modelo

penalty	C	class_weight	AUC	Accuracy	F1	Norm
l1	0.1	None	0.6898	0.8174	0.0000	1.6965
l1	0.1	balanced	0.6883	0.6099	0.3822	1.9592
l1	0.1	{0:1, 1:5}	0.6865	0.5519	0.3783	2.2458
l1	1.0	None	0.6857	0.8174	0.0000	2.5200
l1	10.0	None	0.6852	0.8174	0.0000	2.6077
l2	10.0	None	0.6852	0.8174	0.0000	2.5942
elasticnet	10.0	None	0.6852	0.8174	0.0000	2.6009
elasticnet	1.0	None	0.6849	0.8174	0.0000	2.4589
l2	1.0	None	0.6848	0.8174	0.0000	2.4051
elasticnet	0.1	balanced	0.6848	0.6217	0.3895	1.8098
elasticnet	0.1	None	0.6846	0.8174	0.0000	1.5447
elasticnet	0.1	{0:1, 1:5}	0.6845	0.5434	0.3816	2.1010
l1	1.0	balanced	0.6840	0.6256	0.3894	2.4896
l1	1.0	{0:1, 1:5}	0.6837	0.5597	0.3767	2.5613
elasticnet	1.0	balanced	0.6835	0.6262	0.3898	2.4545
l1	10.0	balanced	0.6835	0.6269	0.3889	2.5446
elasticnet	1.0	{0:1, 1:5}	0.6835	0.5577	0.3768	2.5389
elasticnet	10.0	balanced	0.6835	0.6269	0.3889	2.5408
l2	10.0	balanced	0.6834	0.6269	0.3889	2.5372
elasticnet	10.0	{0:1, 1:5}	0.6833	0.5603	0.3771	2.5916
l1	10.0	{0:1, 1:5}	0.6833	0.5603	0.3771	2.5941
l2	10.0	{0:1, 1:5}	0.6833	0.5603	0.3771	2.5892
l2	1.0	balanced	0.6832	0.6275	0.3919	2.4221
l2	1.0	{0:1, 1:5}	0.6832	0.5558	0.3758	2.5177
l2	0.1	{0:1, 1:5}	0.6830	0.5342	0.3748	2.0016
l2	0.1	balanced	0.6816	0.6217	0.3869	1.7265
l2	0.1	None	0.6796	0.8174	0.0000	1.4979

A.2.4. PARTE E

Table 10. Top variables por modelo ordenadas por $|\tilde{\beta}|$

penalty	C	class_weight	Top variable	$ \tilde{\beta}_m $
l1	1.0	None	Crowdedness	0.942488
l1	10.0	None	Crowdedness	0.941752
elasticnet	10.0	None	Crowdedness	0.941301
l2	10.0	None	Crowdedness	0.940847
l1	0.1	{0:1, 1:5}	Crowdedness	0.940831
l1	0.1	None	Crowdedness	0.939946
l1	0.1	balanced	Crowdedness	0.938502
l1	1.0	{0:1, 1:5}	Crowdedness	0.938194
elasticnet	1.0	None	Crowdedness	0.937863
l1	10.0	{0:1, 1:5}	Crowdedness	0.937857
elasticnet	10.0	{0:1, 1:5}	Crowdedness	0.937690
l2	10.0	{0:1, 1:5}	Crowdedness	0.937523
l1	1.0	balanced	Crowdedness	0.936625
elasticnet	1.0	{0:1, 1:5}	Crowdedness	0.936562
l1	10.0	balanced	Crowdedness	0.936070
elasticnet	10.0	balanced	Crowdedness	0.935781
l2	10.0	balanced	Crowdedness	0.935502
l2	1.0	{0:1, 1:5}	Crowdedness	0.934932
elasticnet	1.0	balanced	Crowdedness	0.933766
l2	1.0	None	Crowdedness	0.933205
l2	1.0	balanced	Crowdedness	0.930888
elasticnet	0.1	{0:1, 1:5}	Crowdedness	0.924455
elasticnet	0.1	balanced	Crowdedness	0.911014
l2	0.1	{0:1, 1:5}	Crowdedness	0.907434
elasticnet	0.1	None	Crowdedness	0.895309
l2	0.1	balanced	Crowdedness	0.882063
l2	0.1	None	Crowdedness	0.850878

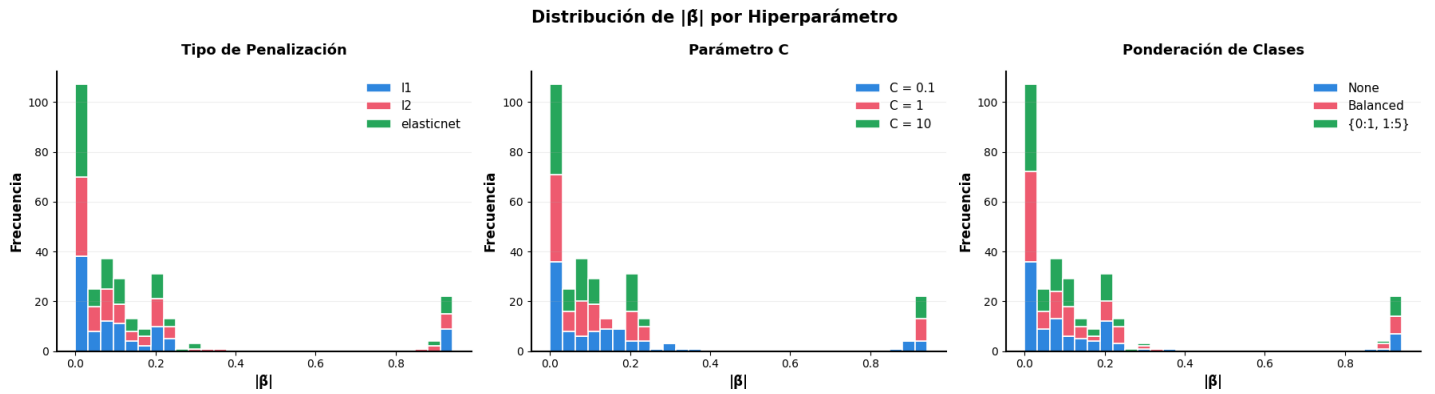


Figure 7. Distribuciones de $|\tilde{\beta}_m|$ dependiendo del tipo de class_weight, penalty y C.

A.3. Pregunta 3

A.3.1. PARTE A

Table 11. Distribución de la variable objetivo en los conjuntos de entrenamiento y prueba.

Tamaño de los conjuntos	X_train: (826, 12), X_test: (92, 12)
Distribución del objetivo (%)	
Conjunto de entrenamiento	1: 55.33%, 0: 44.67%
Conjunto de prueba	1: 55.43%, 0: 44.57%

A.3.2. PARTE B

Table 12. Modelos y sus métricas en los conjuntos de entrenamiento y testeo.

Modelo	Train Acc	Test Acc	Train Prec	Test Prec	Train Rec	Test Rec	Train Spec	Test Spec	Train F1	Test F1	Train FPR	Test FPR	Train FNR	Test FNR	Test AUC
LogisticRegression	0.8571	0.8696	0.8661	0.8980	0.8775	0.8627	0.8320	0.8780	0.8717	0.8800	0.1680	0.1220	0.1225	0.1373	0.9407
Perceptron	0.6283	0.6522	0.8538	0.8800	0.3961	0.4314	0.9160	0.9268	0.5411	0.5789	0.0840	0.0732	0.6039	0.5686	0.8211
KNN	0.7845	0.6848	0.7987	0.7200	0.8162	0.7059	0.7453	0.6585	0.8074	0.7129	0.2547	0.3415	0.1838	0.2941	0.7468
GaussianNB	0.8644	0.8804	0.8726	0.9167	0.8840	0.8627	0.8401	0.9024	0.8783	0.8889	0.1599	0.0976	0.1160	0.1373	0.9536
LDA	0.8535	0.8043	0.8621	0.8511	0.8753	0.7843	0.8266	0.8293	0.8686	0.8163	0.1734	0.1707	0.1247	0.2157	0.9225

A.3.3. PARTE C

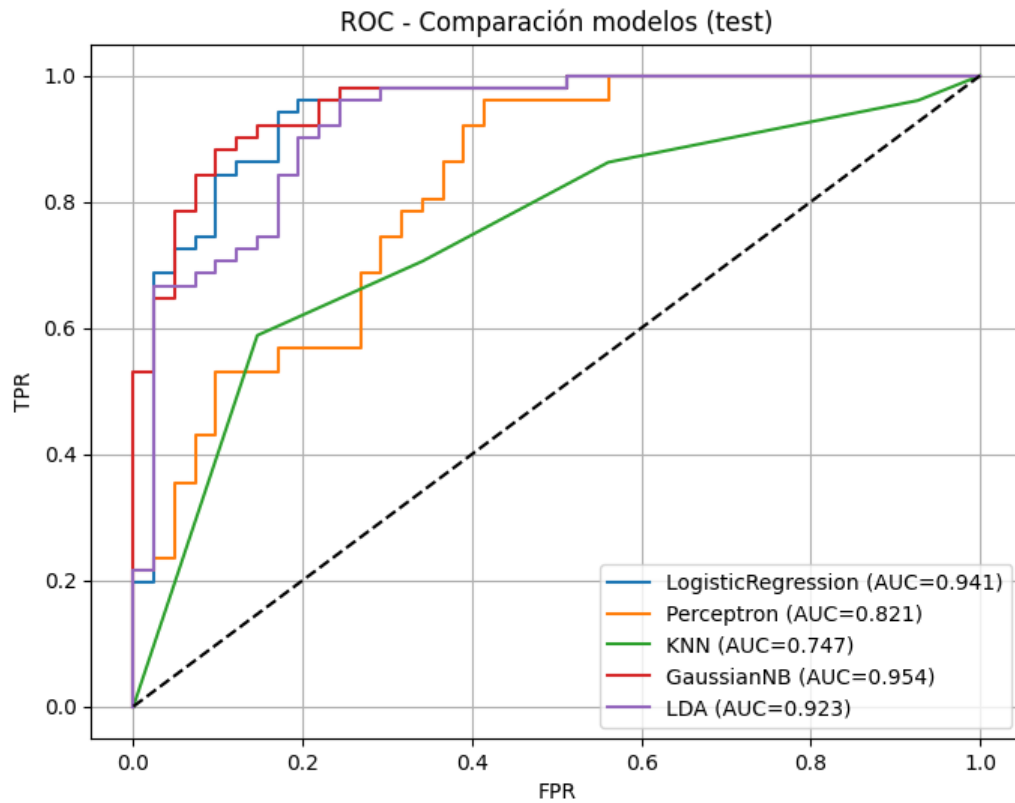


Figure 8. Curva ROC y métrica AUC para todos los modelos.

A.3.4. PARTE D

Table 13. Modelos y sus métricas para distintos conjuntos de variables

AUC			
Modelo	Categorías	Numéricas	Todas
GaussianNB	0.9216	0.9388	0.9665
KNN	0.8967	0.7468	0.7468
LDA	0.9144	0.9374	0.9570
LogisticRegression	0.9216	0.9369	0.9584
Perceptron	0.8840	0.8216	0.8111
Accuracy			
Modelo	Categorías	Numéricas	Todas
GaussianNB	0.8152	0.8587	0.9130
KNN	0.8587	0.6848	0.6848
LDA	0.8043	0.8370	0.9239
LogisticRegression	0.8152	0.8370	0.9239
Perceptron	0.7283	0.6739	0.4891
F1			
Modelo	Categorías	Numéricas	Todas
GaussianNB	0.8090	0.8738	0.9184
KNN	0.8602	0.7129	0.7129
LDA	0.8163	0.8571	0.9307
LogisticRegression	0.8172	0.8598	0.9307
Perceptron	0.7191	0.6512	0.1455

A.3.5. PARTE E

Table 14. Modelos y su desempeño según el tipo de escalador utilizado para el conjunto de todas las variables

AUC			
Modelo	Quantile Normal	Robust	Standard
GaussianNB	0.9484	0.9665	0.9665
KNN	0.9668	0.9512	0.9627
LDA	0.9541	0.9570	0.9570
LogisticRegression	0.9541	0.9579	0.9584
Perceptron	0.8345	0.8900	0.9230
Accuracy			
Modelo	Quantile Normal	Robust	Standard
GaussianNB	0.9022	0.9130	0.9130
KNN	0.9348	0.8913	0.9320
LDA	0.9022	0.9239	0.9239
LogisticRegression	0.9130	0.9239	0.9239
Perceptron	0.7391	0.7935	0.8211
F1			
Modelo	Quantile Normal	Robust	Standard
GaussianNB	0.9109	0.9184	0.9184
KNN	0.9412	0.9000	0.9320
LDA	0.9109	0.9307	0.9307
LogisticRegression	0.9216	0.9307	0.9307
Perceptron	0.7209	0.8288	0.8211